

Agile-практики, DevOps & TDD

От XP до непрерывной интеграции: практические техники



XP: 12 практик качества



DevOps: Культура + CI/CD



TDD: Red-Green-Refactor



BDD: Язык бизнеса (Given-When-Then)

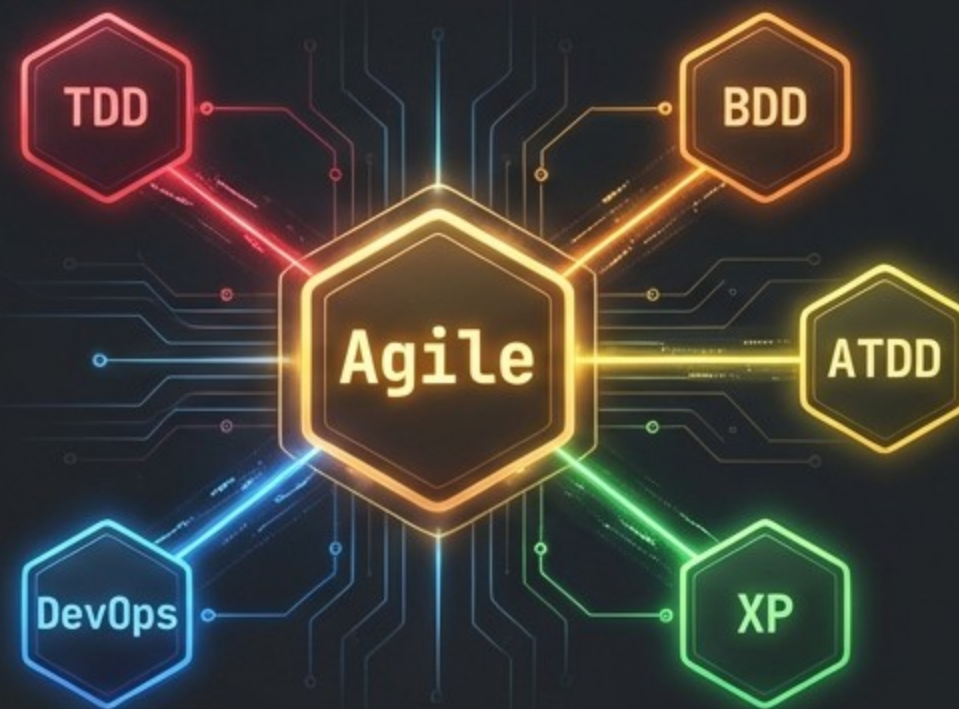


ATDD: Критерии приёмки определяют разработку



Квадранты: Стратегия тестирования

KEY PRINCIPLE: Практики = инструменты.
Выбирайте под задачу, а не под моду. Не все практики подходят для всех проектов.



Extreme Programming (XP): 5 Ценностей

Фундамент для технического совершенства



Коммуникация

// Код = документация



Уважение

// Профессионализм



Простота

// YAGNI



Смелость

// Рефакторинг



Обратная связь

// Быстрые тесты

XP vs Scrum

Scrum – это управление (менеджмент).
XP – это инженерия (код).

Scrum + XP = Мощная комбинация

Context Note:
// XP критичен для Fintech и Medtech.

12 Практик Extreme Programming

Система, где практики усиливают друг друга

```
package java.index;  
  
4  
5 public class MaintainerTotalcreator;  
6  
7  
8  
9  
10  
11  
12  
13  
14  
15  
16  
17  
18  
19  
20  
21  
22  
23  
24  
25  
26  
27  
28  
29  
30  
31  
32
```

PLANNING

1. Planning Game

2. Small Releases

3. 40-Hour Week

DESIGN

4. Metaphor

5. Simple Design

6. Refactoring

IMPLEMENTATION

7. TDD

8. Pair Programming

9. Continuous Integration

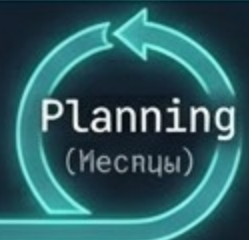
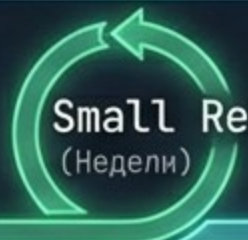
COLLABORATION

10. Code Ownership

11. On-Site Customer

12. Coding Standards

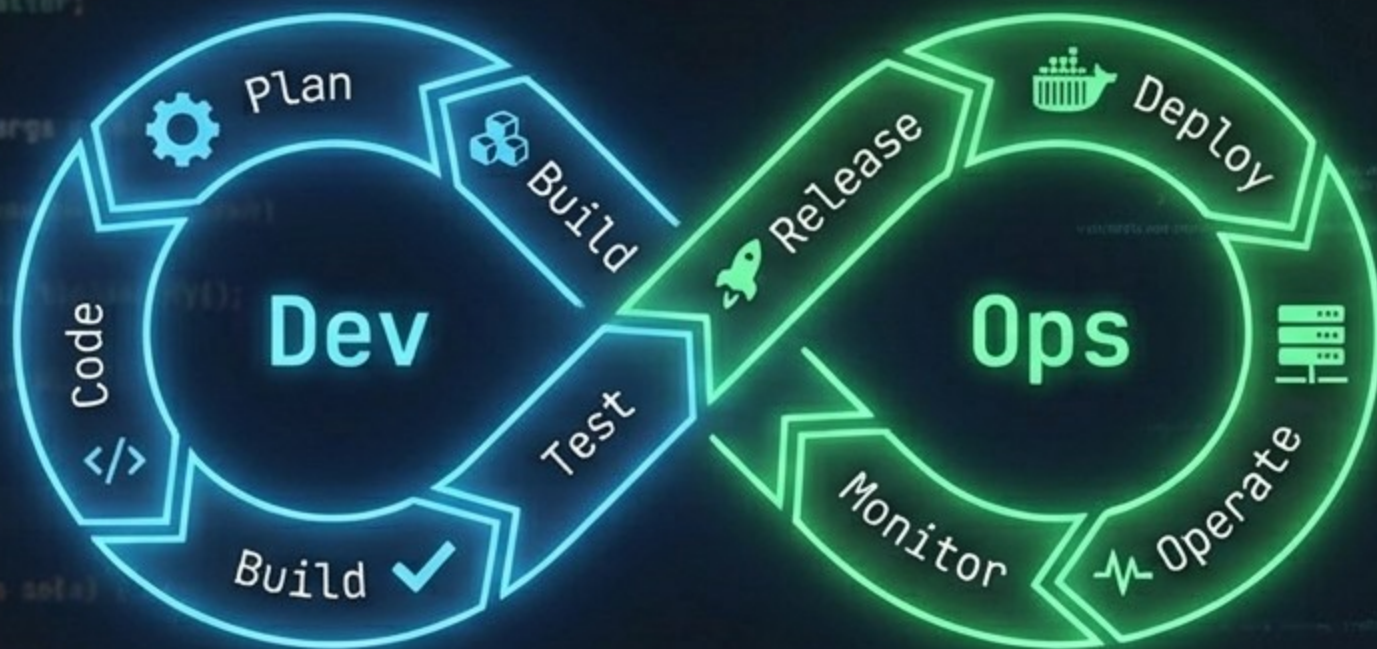
Feedback Loops



Система Feedback Loops: От микро-цикла до макро-цикла.

DevOps: Культура и Инструменты

Объединение Dev + Ops для непрерывной поставки



Dev

Build/Test

Deploy/Monitor



Git



VS Code



SonarQube



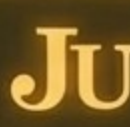
Jenkins



Maven



Selenium



JUnit



Docker



Kubernetes



Terraform



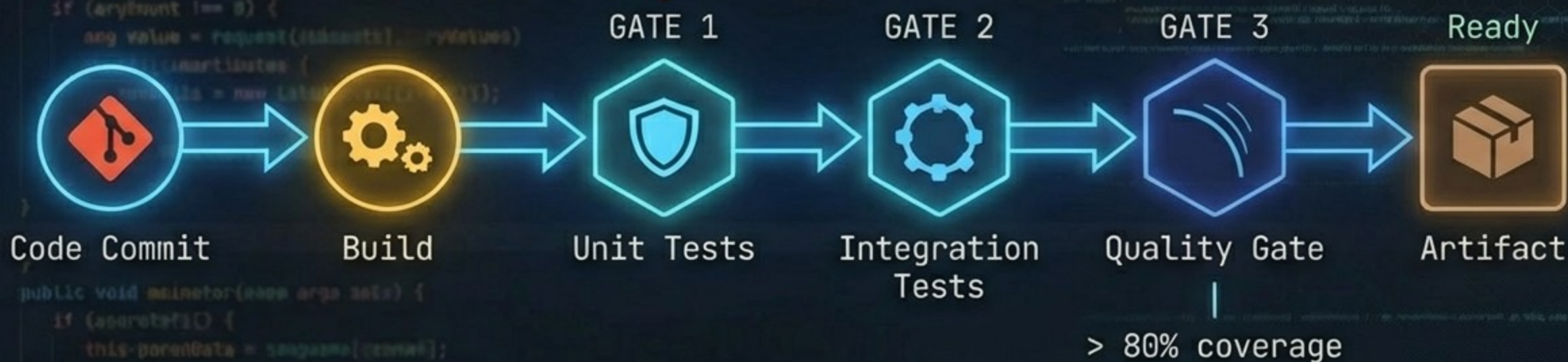
Prometheus

QA в DevOps не просто тестирует. QA автоматизирует тестирование внутри пайплайна. Тесты как код.

Continuous Integration (CI)

Автоматизированная сборка и проверка (5-10 раз в день)

Fail = Stop 🛑



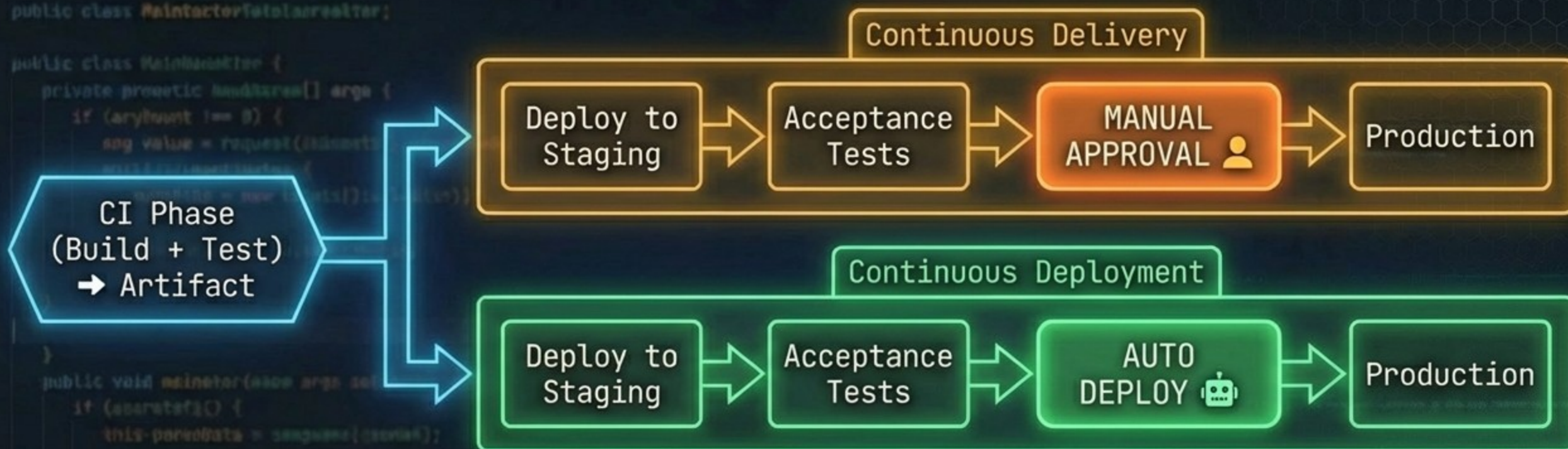
❌ NO CI: Integration Hell 🔥
(Недели на мердж, мистические баги).

✅ WITH CI: Smooth Flow 🌊
(Ошибки ловятся за 5 минут).

Build frequency: 10+/day | Fix time: <30 min | Build success: >95%

Continuous Delivery (CD)

Or Build do Production



Delivery		Всегда готово, релиз по кнопке.		Риск: Средний (Google, Amazon)
----------	--	---------------------------------	--	--------------------------------

Deployment		Релиз автоматический.		Риск: Высокий (Startups)
------------	--	-----------------------	--	--------------------------

QA Insight: QA пишет автотесты, которые служат "воротами качества". Если тест падает — деплой блокируется.

TDD: Red — Green — Refactor

Тесты пишутся ДО кода

● RED

Напиши тест.
Он падает.
(0-5 мин)

● GREEN

Напиши код.
Тест проходит.
(5-10 мин)

● REFACTOR

Улучши код.
(5-15 мин)

Traditional

Code (1 mo)

Test (1 mo)

Fix Bugs (1 mo)

High Debt / 3 Months

TDD

Code+Test (2.5 mo)

Polish (0.5 mo)

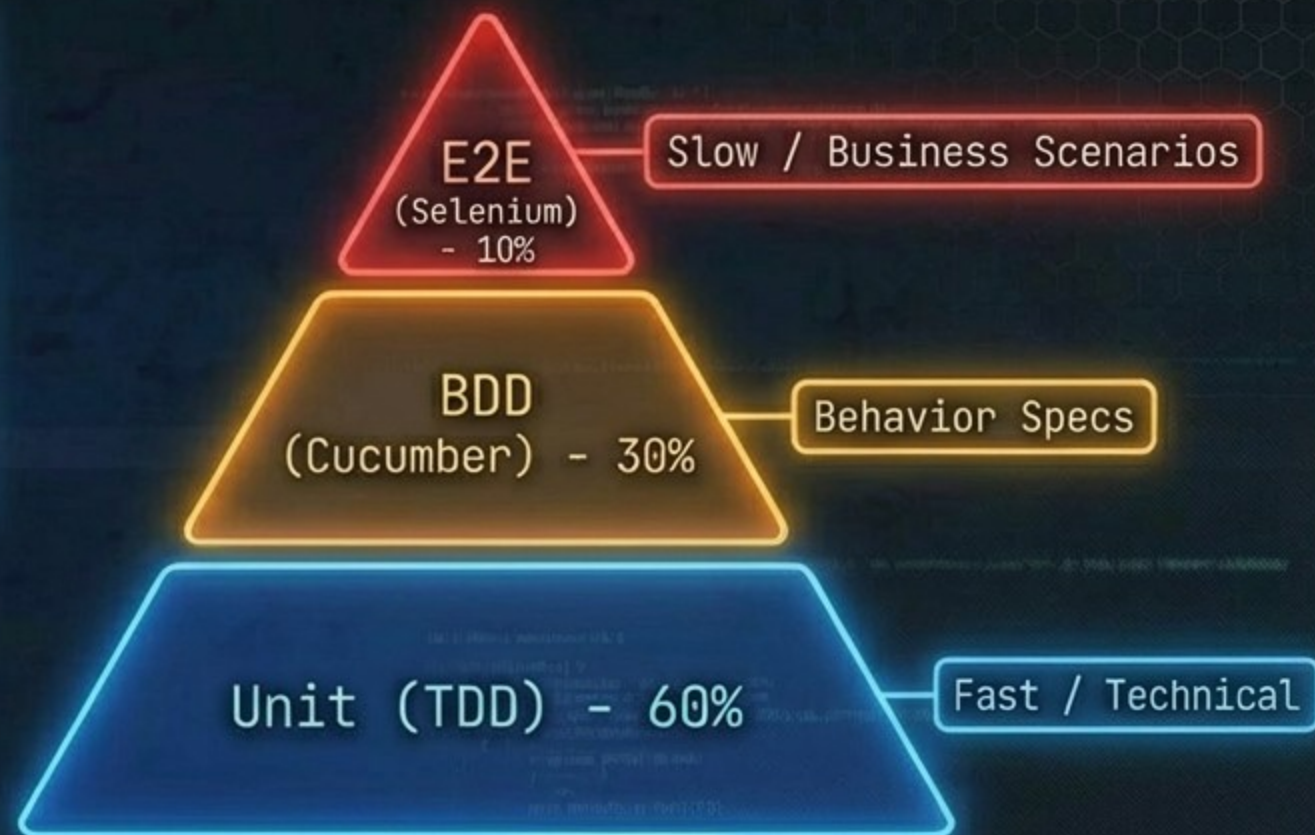
Clean Code / 3 Months

QA Benefit: 90%+ покрытие кода тестами получается автоматически.

BDD: Behavior-Driven Development

Тесты на языке бизнеса (Given-When-Then)

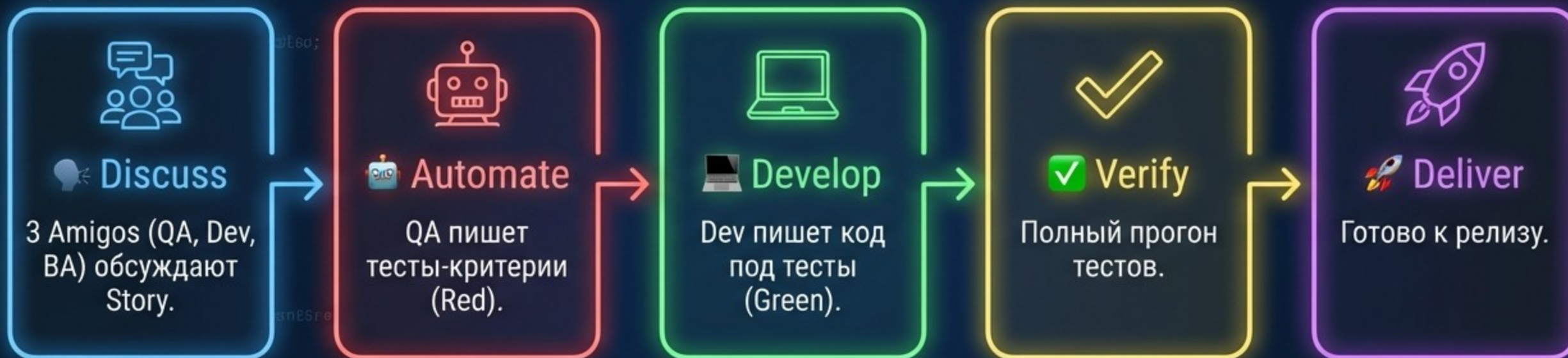
```
1 Feature: Login
2   Scenario: Successful Login
3     ● Given: User is on login page
4     ● When: User enters valid creds
5     ● Then: Dashboard is displayed
```



Tools: Cucumber, SpecFlow, Behave.

ATDD: Acceptance TDD

Сотрудничество QA, Dev и BA до начала работы



Traditional: QA находит баги в конце. Misunderstanding = High.
ATDD: Баги предотвращаются в начале. Misunderstanding = 0.

★ BUILD SUCCESSFUL

Квадранты Тестирования Agile

Классификация по Crispin & Gregory



Strategy: Agile требует внимания ко всем 4 квадрантам! Q1/Q2/Q4 - автоматизация, Q3 - ручное.

Гибриды: Scrumban & Water-Scrum-Fall

Адаптация методологий к реальности

Scrumban (The Sweet Spot)



- ✓ От Scrum: Спринты, Planning.
- ✓ От Kanban: WIP Limits, Pull System.

Best for: Requirements change daily.

Water-Scrum-Fall (The Enterprise Reality)



Use **Scrumban** for startups. Use **Water-Scrum-Fall** only if regulations (Med/Fin) demand it.

Как выбрать подход?

Матрица: Стабильность требований vs Сроки

Stable
Requirements

Dynamic
Requirements

RAD / Lightweight



Prototyping.

Waterfall / V-Model



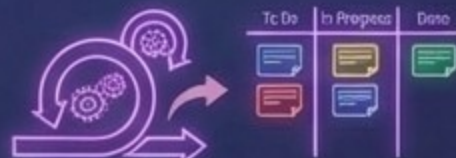
Gov, Med projects.

XP / Kanban



MVP, Hotfixes.

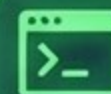
Scrumban / Spiral



Startups scaling up.

Short Time

Long Time



QA Action Plan:

- ✓ 1. Оценить стабильность требований.
- ✓ 2. Выбрать методологию по матрице.
- ✓ 3. Подобрать практики (TDD/CI).

BUILD SUCCESSFUL

BUILD SUCCESSFUL

Трансформация: Traditional → Agile QA

Смена менталитета и инструментов

Traditional: Gatekeeper



Ищет баги после кода.



Блокирует релизы.



Ручные тест-кейсы (Excel).



Метрика: кол-во багов.

Agile/DevOps: Partner



Предотвращает баги (Quality Assurance).



Ускоряет релизы (CI/CD).



Код автотестов (Java/Python).



Метрика: покрытие и cycle time.

Awareness



Adaptation



Optimization



Excellence

Практические советы для QA

Чек-лист выживания в Agile



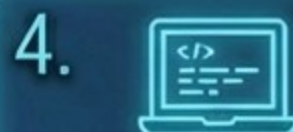
Speak Up:
Не жди конца спринта.



Criteria:
Пиши критерии (Gherkin) ДО кода.



80/20:
80% Automation,
20% Manual.



Code:
Учи основы (Python/Java).



Metrics:
Измеряй Coverage
и Cycle Time.



Sit Together:
Сиди (виртуально) с Dev.



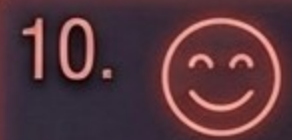
Pipeline:
Запускай тесты в CI.



TDD:
Понимай, как Dev
пишет тесты.



Learn:
Изучай новые тулзы
ежемесячно.



Adapt:
Будь гибким, Agile
— это изменения.

Golden Rule: "Better prevention than cure."

Итоги и Путь Развития

От Junior Tester до QA Architect



“Качество — это ответственность каждого, а не только QA.”